RURAL LEDGER & MACHINERY MANAGEMENT SYSTEM

Tamil-First | Offline-First | Mobile-First

FULL SDLC PROJECT DOCUMENTATION

Software Requirements | System Design | Development Plan | QA | Deployment

Project Name	Rural Ledger & Machinery Management System
Version	1.0.0 — Final
Prepared For	Rural Entrepreneur Client — Tamil Nadu, India
Document Type	Full SDLC Documentation (Phase 1 – Phase 6)
Date	April 2026
Classification	Client Delivery Document

This document is the authoritative specification, design, and delivery plan for the Rural Ledger & Machinery Management System. All development work must strictly follow this document.

PHASE 1 Project Charter & Planning Document

1.1 Project Overview

The Rural Ledger & Machinery Management System is a purpose-built digital platform designed specifically for a Tamil Nadu-based rural entrepreneur who manages multiple agricultural business activities. These include a harvester hiring service, tractor and machinery rentals, own-land farming operations, worker management, and informal lending. The system replaces a fragmented, error-prone notebook-based tracking system with a structured, reliable, and easy-to-use digital solution.

The platform is engineered with three foundational priorities: Tamil-language-first user interface to accommodate native-speaking users, offline-first data storage to ensure uninterrupted operation in areas with limited internet connectivity, and a mobile-first design to support the primary devices used in the field.

1.2 Problem Statement

The client currently manages all business records using handwritten notebooks. This method introduces the following documented problems:

- Frequent arithmetic errors in income, expense, and profit calculations across harvester, rental, and farming activities.
- No consolidated view of daily, weekly, or monthly profitability across all business lines.
- Diesel usage and machine hour tracking are inaccurate because sessions are noted informally and totalled manually.
- Worker salary, advance, and bonus records are scattered across multiple notebooks and prone to disputes.
- Farmer-wise transaction history is unavailable, making it impossible to verify billing or payment disputes.
- The admin cannot generate reports or summaries without manually adding up notebook entries.
- Data is physically vulnerable — notebooks can be lost, damaged, or become illegible.

1.3 Solution Overview

The solution is a modular, single-page web application (SPA) built on React (Vite) with a local-first data persistence strategy using browser LocalStorage. The application is structured around eight distinct business modules that mirror the client's real-world operations. All calculations are automated, data is stored instantly on the device, and the interface uses Tamil labels throughout to maximise accessibility.

1.4 Stakeholder Analysis

Stakeholder	Role	Primary Interest	Interaction Level
Business Entrepreneur	Admin / Owner	Full system control, accurate reports	Daily — Full access
Machine Driver	Driver	Quick, simple log entry	Daily — Data entry only
Village Farmer	Farmer	View own billing and payment history	On-demand — Read only
Workers	Indirect	Accurate salary records	None (admin manages)

1.5 User Role Definitions

Admin (Entrepreneur)

The admin is the primary user and system owner. This role has unrestricted access to all modules, including data creation, editing, deletion, pricing configuration, and report generation. The admin operates from both a mobile phone in the field and a laptop at home. The admin is also responsible for creating and managing Driver and Farmer accounts.

Driver

The driver is a field operator who uses the system exclusively on a mobile device. The driver's sole function is to enter machine usage logs, recording which machine was used, for which farmer, the work sessions (start and end times), and the diesel consumed. Drivers do not have access to financial data, pricing, or other farmers' records. Driver accounts are created and password-managed by the Admin.

Farmer (Read-Only)

The farmer accesses the system to view their own transaction history — machine hours worked, rent charged, and payment status. This user has zero data entry capability. A farmer identifies themselves using their name or farmer ID with no password required, as financial data sensitivity is low and ease of access is the priority. The farmer view is intentionally simple and Tamil-first.

1.6 Operating Environment & Constraints

Constraint	Description	Design Response
Internet Availability	Unstable or absent in rural	All data stored locally; no API dependencies

	areas	
Device Type	Primarily Android mobile phones	Mobile-first responsive layout, large touch targets
User Literacy	Some users have limited reading literacy	Tamil labels, icon-assisted UI, minimal navigation
Language	Tamil is the primary language	Every field label includes Tamil text
Technical Skill	Non-technical users	Large buttons, simple forms, confirmation messages
Data Entry Timing	Logs entered during or after work	Multi-session support, no real-time requirement

1.7 Project Scope

In Scope

- Dashboard with real-time income, expense, and profit summary
- User management (Farmers, Drivers, Workers — CRUD operations)
- Finance ledger: household expense tracking and informal lending records
- Own farming: admin's personal agriculture expense and income tracking
- Machinery rental: tractor (per hour), plough tractor (per hour), trailer (per trip)
- Harvester management: tyre and track machines, session-based billing, wet field surcharge
- Worker management: work entry, salary, advance, bonus, and net payable calculation
- Driver logs: multi-session time entry, diesel tracking (rupees or litres mode)
- Farmer view: read-only personal record portal
- Monthly reports and machine-wise profit reports
- Data export as JSON (manual backup) and PDF report generation
- WhatsApp share capability (text or PDF link)
- Progressive Web App (PWA) manifest for Add-to-Home-Screen capability

Out of Scope (Future Phases)

- Cloud synchronisation across multiple devices
- Online payment integration
- Multi-entrepreneur / multi-tenant support
- OTP-based authentication
- Advanced analytics and forecasting
- Backend server or REST API

1.8 Project Timeline

Phase / Day	Focus Area	Deliverable
Day 1	Project setup, base structure	Running app with folder layout and base components
Day 2	Driver entry UI	Complete visual form, Tamil labels, mobile layout
Day 3	Time calculation logic	Accurate session-based hour calculations
Day 4	Diesel logic + data save	Data persists across page reloads via LocalStorage
Day 5	Harvester module (core)	Rent calculation linked to driver logs
Day 6	Machinery rental module	Tractor/trailer/water rental working
Day 7	Worker management	Worker records with salary and advance net-payable
Day 8	Dashboard	Live income, expense, profit summary from all modules
Day 9	Reports + PWA setup	Monthly/machine reports, offline-ready PWA
Day 10	Export, testing, polish	PDF export, WhatsApp share, full end-to-end testing

1.9 Risk Register

Risk	Probability	Impact	Mitigation Strategy
Data loss if browser storage is cleared	Medium	High	JSON export/import backup feature
Low usability for non-literate users	Medium	High	Tamil UI, large elements, on-site training
Internet unavailability during use	High	High	Full offline-first architecture
Calculation errors in complex logic	Low	High	Unit test each calculation function
Device incompatibility (old Android)	Medium	Medium	Test on low-end devices before delivery
Developer scope creep	Medium	Medium	Strict MVP definition, no unapproved changes

1.10 Success Criteria

The system will be considered successfully delivered when all of the following conditions are verified:

1. A driver can independently enter a machine usage log (with sessions and diesel) without assistance within 2 minutes.

2. The harvester rent is automatically calculated from logged hours with no manual arithmetic required from the admin.
3. Data entered offline is available without any loss after the app is closed and reopened.
4. The admin can view total income, total expense, and net profit for any given month from the dashboard.
5. A farmer can access their personal transaction history using only their name or ID.
6. The admin can export all data as a JSON backup file and restore it successfully.
7. The app passes usability testing with a real driver user and a real farmer user in village conditions.

PHASE 2 Software Requirements Specification (SRS)

2.1 Purpose of This Document

This Software Requirements Specification (SRS) defines, in precise and unambiguous terms, the complete functional and non-functional requirements for the Rural Ledger & Machinery Management System. Every requirement stated here is directly traceable to a business need identified in Phase 1. This document serves as the binding contract between the analyst and the development team — no behaviour should be implemented that is not specified here, and no specified behaviour should be omitted.

2.2 System Modules Overview

Module ID	Module Name	Primary User	Core Purpose
M01	Dashboard	Admin	Aggregated income, expense, and profit summary
M02	User Management	Admin	CRUD for Farmers, Drivers, Workers
M03	Finance Ledger	Admin	Household expenses and informal lending
M04	Own Farming	Admin	Personal agriculture expense and income
M05	Machinery Rental	Admin / Driver	Tractor, plough, trailer rental billing
M06	Harvester Management	Admin	Harvester billing, expenses, profit per job
M07	Worker Management	Admin	Worker records, salary, advance, net payable
M08	Driver Logs	Driver / Admin	Machine session logs, diesel tracking
M09	Farmer View	Farmer	Read-only personal transaction history

2.3 Authentication Requirements

Role	Login Method	Password	Access Granted
Admin	Single password screen	Set during first launch	All modules — full CRUD
Driver	Select name from list + PIN	4-digit PIN set by Admin	Driver Log entry only
Farmer	Select name from list	None required	Own records — read only

Security Note: Farmer access requires no password because the farmer's data (billing and payment records) is already shared with them in person. The risk of cross-farmer data exposure is mitigated by using unique Farmer IDs (not names) as the internal reference key throughout the system.

2.4 Module-Wise Functional Requirements

M01 — Dashboard Module

The dashboard is the home screen presented to the Admin immediately after login. It must display a consolidated real-time summary derived from all other modules.

Display Element	Data Source	Calculation Method
Total Income (This Month)	Harvester + Rental + Farming	Sum all income records for current month
Total Expense (This Month)	All expense records with source tag	Sum all expense records for current month
Net Profit (This Month)	Derived	Total Income minus Total Expense
Today's Machine Sessions	Driver Logs	Count of log entries for today's date
Today's Diesel Cost	Driver Logs	Sum of diesel.total from today's logs
Pending Lendings	Finance Ledger	Count of lending records with status = Pending

M02 — User Management Module

This module is Admin-only and provides full create, read, update, and delete (CRUD) capability for the three entity types used across all modules.

Farmer Entity Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Format: F001, F002...	—
name	String	Required, max 60 chars	விவசாயி பெயர்
village	String	Required	கிராமம்
phone	String	Optional, numeric, 10 digits	தொலைபேசி

type	Enum	neighbour | normal	வகை

Driver Entity Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Format: D001, D002...	—
name	String	Required, max 60 chars	ஓட்டுநர் பெயர்
phone	String	Optional, 10 digits	தொலைபேசி
pin	String	4-digit numeric, set by Admin	கடவுச்சொல்

Worker Entity Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Format: W001, W002...	—
name	String	Required, max 60 chars	தொழிலாளர் பெயர்
village	String	Optional	கிராமம்
phone	String	Optional, 10 digits	தொலைபேசி

M03 — Finance Ledger Module

This module tracks the admin's personal household finances and informal lending activity. These records are separate from business income and expense.

Home Expense Sub-Module

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
date	Date	Required, not future	தேதி
category	Enum	Food / Medical / Transport / Utilities / Other	வகை
description	String	Optional, max 100 chars	விளக்கம்
amount	Number	Required, > 0	தொகை (₹)

Lending Sub-Module

Field	Type	Validation	Tamil Label

id	String (auto)	Auto-generated	—
personName	String	Required	பெயர்
village	String	Optional	கிராமம்
amountGiven	Number	Required, > 0	கொடுத்த தொகை (₹)
dateLent	Date	Required	கொடுத்த தேதி
amountReturned	Number	Default 0	திரும்ப பெற்ற தொகை (₹)
dateReturned	Date	Optional	திரும்பிய தேதி
status	Enum	pending | partial | returned	நிலை
notes	String	Optional	குறிப்பு

M04 — Own Farming Module

This module records the admin's personal agricultural operations. It is entirely separate from the harvester business — it tracks the costs and income of the admin's own land cultivation.

NOTE — Reserved Farmer ID for Own Farm: A reserved Farmer record with ID F000 and name 'Own Farm' (சொந்த நிலம்) is automatically created on first launch. When a driver logs machine hours for the admin's own land, they select 'Own Farm' from the farmer dropdown. This prevents double-counting and keeps all machine usage in the unified Driver Logs module.

Own Farming Expense Fields

Category	Input Type	Notes	Tamil Label
Diesel	Amount (₹)	Direct rupee entry	டீசல்
Maintenance	Amount (₹)	Machine maintenance costs	பராமரிப்பு
Spare Parts	Amount (₹)	Replacement parts	உதிரி பாகங்கள்
Driver Salary	Amount (₹)	Daily or per-trip	ஓட்டுநர் சம்பளம்
Worker Salary	Amount (₹)	Field workers	தொழிலாளர் சம்பளம்
Food Expense	Amount (₹)	Multiple per day allowed (Morning/Afternoon/Evening)	உணவு செலவு
Motor Maintenance	Amount (₹)	Irrigation motor costs	மோட்டார் பராமரிப்பு

Own Farming Income Fields

Field	Type	Calculation	Tamil Label
Harvest Date	Date	Required	அறுவடை தேதி

Crop Type	String	e.g., Paddy, Sugarcane	பயிர் வகை
Number of Bags	Number	Required, > 0	மூட்டைகளின் எண்ணிக்கை
Price Per Bag (₹)	Number	Required, > 0	ஒரு மூட்டை விலை
Total Income (auto)	Computed	Bags × Price Per Bag	மொத்த வருமானம்

M05 — Machinery Rental Module

This module records income from renting out machines to farmers. Each rental entry is linked to a specific farmer and machine type.

Machine	Billing Unit	Rate Set By	Tamil Name
Tractor	Per Hour	Admin (editable)	டிராக்டர்
Plough Tractor	Per Hour	Admin (editable)	உழவு டிராக்டர்
Trailer	Per Trip	Admin (editable)	ட்ரெய்லர்
Water Rental	Per Season	Admin (per-farmer assignment)	தண்ணீர் குத்தகை

Rental Entry Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
date	Date	Required	தேதி
farmerId	Reference	Must exist in Farmers collection	விவசாயி
machineType	Enum	tractor | plough_tractor | trailer | water	இயந்திர வகை
unit	Enum	hour | trip | season	அலகு
quantity	Number	Required, > 0	அளவு
ratePerUnit	Number	Required, > 0 (fetched from pricing)	ஒரு அலகு விலை
totalAmount	Computed	quantity × ratePerUnit	மொத்த தொகை
paymentStatus	Enum	pending | paid | partial	பணம் நிலை
amountPaid	Number	Default 0	செலுத்திய தொகை

M06 — Harvester Management Module

This is the core revenue module. It connects directly to Driver Logs to retrieve total machine hours worked for a farmer, applies the configured pricing, and computes the final bill. It also tracks all harvester-specific expenses to compute per-job profitability.

Pricing Logic (Resolved)

Billing Formula: Total Bill = (Total Hours from Driver Logs × Rate Per Hour) − Discount Wet Field Surcharge: When wetField = true for a Tyre-type harvester, a higher rate (configured by admin) is applied instead of the standard rate. Track-type harvesters always use a fixed high rate, editable only by Admin.

Harvester Job Record Fields

Field	Type	Validation / Notes	Tamil Label
id	String (auto)	Auto-generated	—
farmerId	Reference	Must exist in Farmers collection	விவசாயி
village	String	Auto-populated from farmer record	கிராமம்
season	String (free-text)	e.g., 'Kuruvai 2026', 'Samba 2025-26'	பருவம்
machineType	Enum	tyre | track	இயந்திர வகை
wetField	Boolean	Applicable only if machineType = tyre	நீர் நிலம்
ratePerHour	Number	Fetched from HarvesterPricing, editable	மணி நேர விலை
linkedLogIds	Array	IDs of Driver Logs linked to this job	—
totalHours	Computed	Sum of hours from linked Driver Logs	மொத்த மணி நேரம்
grossAmount	Computed	totalHours × ratePerHour	மொத்த வாடகை
discount	Number	Default 0, Admin-set	தள்ளுபடி
finalAmount	Computed	grossAmount − discount	இறுதி தொகை
paymentStatus	Enum	pending | paid | partial	பணம் நிலை
amountPaid	Number	Default 0	செலுத்திய தொகை

Harvester Job Expense Fields

Expense Category	Input	Tamil Label

Diesel	Amount (₹)	டீசல்
Worker Salary	Amount (₹)	தொழிலாளர் சம்பளம்
Spare Parts	Amount (₹)	உதிரி பாகங்கள்
Maintenance	Amount (₹)	பராமரிப்பு
Mechanic Cost	Amount (₹)	மெக்கானிக் கட்டணம்
Driver Food	Amount (₹)	ஓட்டுநர் உணவு

M07 — Worker Management Module

This module manages all worker records and their payment entries. Workers may be paid a salary with deductions for advance payments and additions for bonuses and extra work.

Worker Payment Formula (Resolved)

Net Payable = (Base Salary + Bonus + Extra Amount) − Advance Paid All five fields are independently editable. Advance is subtracted because it has already been given to the worker prior to the salary payment. Net Payable is always computed — it is never entered manually.

Work Entry Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
workerId	Reference	Must exist in Workers collection	தொழிலாளர்
date	Date	Required	தேதி
workType	Enum + Free-text	Dropdown list; last item is 'Other' (manual entry)	வேலை வகை
baseSalary	Number	Required, >= 0	அடிப்படை சம்பளம் (₹)
advance	Number	Default 0	முன்பணம் (₹)
bonus	Number	Default 0	போனஸ் (₹)
extraAmount	Number	Default 0	கூடுதல் தொகை (₹)
netPayable	Computed	(baseSalary + bonus + extraAmount) − advance	நிகர செலுத்தவேண்டியது (₹)
notes	String	Optional	குறிப்பு

M08 — Driver Logs Module

This is the foundational data-entry module that feeds information into Harvester Management and the Dashboard. It must be the simplest and most reliable module in the system since it is used by drivers in the field.

Driver Log Entry Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
driverId	Reference	Must exist in Drivers collection	ஓட்டுநர்
farmerId	Reference	Must exist (includes F000 for Own Farm)	விவசாயி
machineType	Enum	harvester_tyre | harvester_track | tractor | plough_tractor | trailer	இயந்திர வகை
date	Date	Required, defaults to today	தேதி
sessions	Array	Minimum 1 session required	வேலை நேரம்
diesel.mode	Enum	rupees | litres	டீசல் முறை
diesel.value	Number	Required if diesel used, > 0	டீசல் அளவு
diesel.pricePerLitre	Number	Required only if mode = litres	லிட்டர் விலை
diesel.total	Computed	If rupees: value. If litres: value × pricePerLitre	டீசல் செலவு (₹)
source	Enum	driver_log (fixed)	—

Session Sub-Record Fields

Field	Type	Validation
start	Time (HH:MM)	Required
end	Time (HH:MM)	Required; must be after start time
durationHours	Computed	Calculated on save; negative-guard applied for midnight crossover

Overnight Session Handling (Resolved)

> *If end time is numerically before start time (e.g., start 23:00, end 01:00), the system adds 24*

hours to the end time before calculating duration. This handles late-night to early-morning sessions without error. Formula: duration = end < start ? (end + 24h) - start : end - start

M09 — Farmer View Module

This is a read-only portal for farmers. The farmer selects their name from a dropdown list (no password required) and sees a date-ordered history of all activity linked to their Farmer ID across all modules.

Data Displayed	Source Module	Tamil Label
Date of work	Driver Logs	தேதி
Machine used	Driver Logs	இயந்திரம்
Hours worked	Driver Logs (computed)	மணி நேரம்
Harvester rent charged	Harvester Management	வாடகை (₹)
Rental charges	Machinery Rental	குத்தகை (₹)
Payment status	Harvester / Rental	பணம் நிலை
Balance due	Computed	Total charged − Amount paid

2.5 Business Logic Rules

Time and Session Calculations

- Duration per session = End Time − Start Time (with overnight guard: if result is negative, add 24 hours).
- Total hours for a log entry = Sum of all session durations in that entry.
- Total hours for a farmer (per job) = Sum of all linked Driver Log hours for that farmerId and machineType within the job period.
- Sessions within a single log entry may overlap only if they represent different machines (system does not validate inter-session gaps).

Harvester Billing Logic

- Admin opens a Harvester Job record for a specific farmer and season.
- Admin links one or more Driver Log entries to this job using their log IDs.
- System auto-computes total hours from the linked logs.
- Rate is fetched from the HarvesterPricing configuration for the matching farmerId + machineType + season.
- Final Bill = (Total Hours × Rate Per Hour) − Discount.
- If wetField = true on a Tyre machine, the wet field rate is applied instead of the standard rate.

Expense Source Tagging (Resolved)

Every expense record must carry a 'source' field with one of these values: driver_log | harvester | rental | worker | own_farm | home_expense | lending This enables the Dashboard to correctly segregate business expenses from home expenses, and prevents double-counting. Business profit is calculated only from business-sourced income and expenses.

2.6 Non-Functional Requirements

Category	Requirement	Acceptance Threshold
Performance	Page/screen load time	Under 2 seconds on a 2-year-old Android phone
Offline	All features functional without internet	Zero dependency on network for read/write
Reliability	No data loss on app close/reopen	100% data persistence via LocalStorage
Usability	Driver can log entry without help	Achievable in under 2 minutes first time
Accessibility	Tamil labels on all input fields	Every field has a Tamil label
Compatibility	Android Chrome (primary)	Chrome 90+; desktop Chrome as secondary
Maintainability	Modular code structure	Each module in its own folder; logic in services/

2.7 Validation Rules

Field / Rule	Condition	Error Message
End Time	Must be after Start Time (after overnight guard)	முடிவு நேரம் தொடக்க நேரத்திற்கு பிறகு இருக்க வேண்டும்
Diesel Value	Must be > 0	டீசல் மதிப்பு சரியாக இருக்க வேண்டும்
Price Per Litre	Required if diesel mode = litres; must be > 0	லிட்டர் விலை தேவை
Farmer Reference	farmerId must exist in Farmers collection	விவசாயி கிடைக்கவில்லை
Required Fields	All marked required fields must be filled	தேவையான புலங்களை நிரப்பவும்
Net Payable	May be negative (over-advance given) — display warning	முன்பணம் அதிகமாக கொடுக்கப்பட்டது
Amount Values	All monetary fields must	தொகை சரியாக இருக்க வேண்டும்

	be >= 0	

PHASE 3

System Design Document (SDD)

3.1 Architecture Overview

The system uses a Single-Page Application (SPA) architecture built on React with Vite as the build tool. There is no backend server, no REST API, and no database server. All data is persisted in the browser's LocalStorage, structured as named JSON collections. The application is entirely self-contained — it can be served from a static file host (Vercel or Netlify) and will function fully offline after the first load, using a PWA service worker to cache the application shell.

Architecture Stack: React 18 (Vite) → Component Layer → Service Layer → LocalStorage / IndexedDB Future upgrade path: Replace LocalStorage calls in storage.js with API calls — zero change required in components or pages.

3.2 Folder Structure

This structure must be followed without deviation. All new files must be placed in the appropriate directory.

```
src/
  components/
    common/
      Button.jsx           — Reusable styled button
      InputField.jsx       — Label (Tamil+English) + input
      TamilLabel.jsx       — Dual-language label renderer
      SelectField.jsx      — Dropdown with Tamil labels
      ConfirmDialog.jsx    — Delete confirmation modal
      Badge.jsx            — Status indicator (paid/pending)
    dashboard/
      SummaryCard.jsx      — Single stat display card
    driver/
      SessionEntry.jsx     — Single time session row
      SessionList.jsx      — All sessions for one log
      DieselInput.jsx      — Rupees / Litres toggle input
    harvester/
      MachineSelector.jsx
      RateDisplay.jsx
      JobExpenseForm.jsx
    rental/
      RentalForm.jsx
    worker/
      WorkerForm.jsx
      WorkTypeDropdown.jsx
    farmer/
      FarmerHistory.jsx
```

```
  pages/
    Login.jsx
    Dashboard.jsx
    DriverEntry.jsx
    Harvester.jsx
    Rental.jsx
    Workers.jsx
    FarmerView.jsx
    Finance.jsx
    OwnFarming.jsx
    UserManagement.jsx
    Reports.jsx
  services/
    storage.js             — All LocalStorage read/write
    calculations.js        — All business logic functions
  utils/
    timeUtils.js           — Time parsing and duration
    idGenerator.js         — Auto ID generation (F001, D001...)
    formatters.js          — Currency, date display helpers
  data/
    initialData.js         — Seed data (incl. F000 Own Farm)
    workTypes.js           — Worker work type dropdown list
    machineTypes.js        — Machine enum labels
  App.jsx
  main.jsx
```

3.3 Data Schemas

All collections are stored in LocalStorage under the keys shown. Each record's ID is auto-generated using the idGenerator utility. All collections are plain JSON arrays.

Collection: farmers (key: 'rl_farmers')

```
{
  id:       'F000',                // Reserved: Own Farm
  name:     'Own Farm',             // சொந்த நிலம்
  village:  '',
  phone:    '',
  type:     'own'                   // own | neighbour | normal
}
```

Collection: driverLogs (key: 'rl_driver_logs')

```
{
  id:          'LOG001',
  driverId:    'D001',
  farmerId:    'F001',              // 'F000' for own farm
  machineType: 'harvester_tyre',    // see machine enum
  date:        '2026-04-20',
  sessions: [
```

```
    { start: '09:00', end: '10:45', durationHours: 1.75 },
    { start: '11:30', end: '14:00', durationHours: 2.50 }
  ],
  totalHours:  4.25,              // sum of session durations
  diesel: {
    mode:           'litres',      // 'rupees' | 'litres'
    value:          10,
    pricePerLitre:  102,           // only if mode = litres
    total:          1020           // computed on save
  },
  source: 'driver_log'
}
```

Collection: harvesterJobs (key: 'rl_harvester_jobs')

```
{
  id:             'HJ001',
  farmerId:       'F001',
  village:        'Kaveripattinam',
  season:         'Kuruvai 2026',    // free-text season label
  machineType:    'tyre',             // 'tyre' | 'track'
  wetField:       true,               // false for normal field
  ratePerHour:    2500,               // wet field rate applied
  linkedLogIds:   ['LOG001', 'LOG002'],
  totalHours:     7.75,               // computed from linked logs
  grossAmount:    19375,              // totalHours × ratePerHour
  discount:       500,
  finalAmount:    18875,              // grossAmount - discount
  expenses: [
    { category: 'diesel',  amount: 2000, source: 'harvester' },
    { category: 'worker',  amount: 1500, source: 'harvester' }
  ],
  totalExpense:   3500,
  netProfit:      15375,              // finalAmount - totalExpense
  paymentStatus:  'partial',          // pending | paid | partial
  amountPaid:     10000,
  balance:        8875                // finalAmount - amountPaid
}
```

Collection: rentalEntries (key: 'rl_rentals')

```
{
  id:           'RNT001',
  date:         '2026-04-20',
  farmerId:     'F002',
  machineType:  'tractor',    // tractor | plough_tractor | trailer | water
  unit:         'hour',       // hour | trip | season
  quantity:     3,
  ratePerUnit:  500,
  totalAmount:  1500,         // quantity × ratePerUnit
  paymentStatus: 'pending',
  amountPaid:   0,
```

```
  source:       'rental'
}
```

Collection: workEntries (key: 'rl_work_entries')

```
{
  id:           'WE001',
  workerId:     'W001',
  date:         '2026-04-20',
  workType:     'Field Levelling',  // or 'Other: <custom text>'
  baseSalary:   800,
  advance:      200,
  bonus:        100,
  extraAmount:  50,
  netPayable:   750,   // (800 + 100 + 50) - 200 = 750
  notes:        '',
  source:       'worker'
}
```

Collection: expenses (key: 'rl_expenses')

```
{
  id:       'EXP001',
  date:     '2026-04-20',
  category: 'diesel',
  amount:   2000,
  // source tags for dashboard segregation:
  source:   'own_farm'  // own_farm | home_expense | harvester | etc.
}
```

Collection: lendingRecords (key: 'rl_lending')

```
{
  id:             'LND001',
  personName:     'Selvan',
  village:        'Papanasam',
  amountGiven:    5000,
  dateLent:       '2026-03-15',
  amountReturned: 2000,
  dateReturned:   '2026-04-01',
  status:         'partial',  // pending | partial | returned
  notes:          ''
}
```

3.4 Service Layer Design

storage.js — Data Persistence Service

```
// Save entire collection
export function saveData(key, data) {
  localStorage.setItem(key, JSON.stringify(data));
}

// Get entire collection (returns [] if not found)
export function getData(key) {
  return JSON.parse(localStorage.getItem(key)) || [];
}

// Add single record to a collection
export function addRecord(key, record) {
  const list = getData(key);
  list.push(record);
  saveData(key, list);
}

// Update a record by ID
export function updateRecord(key, id, updated) {
  const list = getData(key);
  const idx = list.findIndex(r => r.id === id);
  if (idx !== -1) { list[idx] = updated; saveData(key, list); }
}

// Delete a record by ID
export function deleteRecord(key, id) {
  const list = getData(key).filter(r => r.id !== id);
  saveData(key, list);
}
```

calculations.js — Business Logic Service

```
// Duration between two time strings, with overnight guard
export function getDuration(start, end) {
  const base = '2000-01-01T';
  let s = new Date(base + start);
  let e = new Date(base + end);
  if (e <= s) e = new Date(e.getTime() + 24 * 60 * 60 * 1000);
  return (e - s) / (1000 * 60 * 60);  // returns decimal hours
}

// Total hours from sessions array
export function getHours(sessions) {
  return sessions.reduce((sum, s) => sum + getDuration(s.start, s.end), 0);
}

// Diesel total based on mode
export function getDieselCost(mode, value, pricePerLitre) {
  return mode === 'litres' ? value * pricePerLitre : value;
```

```
}

// Harvester rent calculation
export function calcRent(hours, rate, discount = 0) {
  return Math.max(0, (hours * rate) - discount);
}

// Worker net payable
export function calcNetPay(salary, bonus, extra, advance) {
  return (salary + bonus + extra) - advance;
}

// Profit
export function calcProfit(income, expense) {
  return income - expense;
}
```

3.5 UI Design Patterns

Mobile-First Rules

- Minimum touch target size: 44px height for all interactive elements.
- Font size minimum: 16px (32 DXA in document terms) for all body text.
- Form layout: single-column vertical stack; no side-by-side inputs on mobile.
- Button text: large, clear action words in Tamil with English in parentheses.
- Colour use: minimal palette; green for income/profit, red for expense/loss, navy for headings.

Tamil Label Convention

Every input field renders its label in the format: 'English Label (தமிழ் பெயர்)'. The TamilLabel.jsx component handles this rendering automatically given two props: english and tamil.

Driver Entry Screen Layout

```
Screen: Driver Entry (ஓட்டுநர் பதிவு)
──────────────────────────────────────
[ Machine Type Dropdown ]
[ Farmer Name Dropdown  ]
──────────────────────────────────────
Work Sessions (வேலை நேரம்):
[ Start Time ] [ End Time ] [ Duration ]
[ + Add Another Session ]
──────────────────────────────────────
Diesel (டீசல்):
( ) Rupees (ரூபாய்)   ( ) Litres (லிட்டர்)
[ Amount input ]  [ if litres: price/litre ]
──────────────────────────────────────
[ SAVE LOG (பதிவு செய்க) ]
```

3.6 Data Flow Diagrams

Primary Flow: Driver Log to Dashboard

```
Driver enters log
   → Sessions + Diesel saved to rl_driver_logs
         |
         ↓
Admin opens Harvester Job for farmer
   → Links relevant Driver Log IDs
   → System computes totalHours from linked logs
   → Applies ratePerHour from harvesterPricing config
   → Computes: grossAmount, finalAmount, netProfit
   → Saved to rl_harvester_jobs
         |
         ↓
Dashboard reads all collections
   → Sums income (harvester + rental + own farm harvest)
   → Sums expenses (source != home_expense, lending)
   → Displays: totalIncome, totalExpense, netProfit
```

3.7 PWA & Offline Architecture (Resolved)

The application must include a PWA manifest and a service worker to ensure that after the first load, the app shell (HTML, JS, CSS) is cached and available without internet. This is a hard requirement for real village usage.

Files Required

- public/manifest.json — App name, icons, theme colour, display mode (standalone)
- src/sw.js — Service worker caching the app shell using Cache API
- Vite PWA plugin (vite-plugin-pwa) — Recommended for automatic SW generation

```
// vite.config.js
import { VitePWA } from 'vite-plugin-pwa';
export default {
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      manifest: {
        name: 'கிராம கணக்கு',
        short_name: 'கணக்கு',
        theme_color: '#1B3A6B',
        background_color: '#FFFFFF',
        display: 'standalone',
        icons: [{ src: '/icon-192.png', sizes: '192x192', type: 'image/png' }]
      }
    })
  ]
```

```
};
```

PHASE 4 Development Execution Plan

4.1 Development Strategy

Development follows a strict module-by-module progression. No module is started until the previous module is functionally complete and locally tested. This prevents integration surprises and ensures each piece works correctly before it becomes a dependency for the next piece. The development flow for each module is always: UI → State Management → Business Logic → LocalStorage Save → Manual Test.

4.2 Development Priority Order

Priority	Module	Reason
1	Project Setup + Base Components	Foundation for all other modules
2	Driver Entry UI (no logic)	UI must be visually correct before logic is wired
3	Driver Logic + Time Calculation	Core data that feeds harvester and dashboard
4	Diesel Logic + LocalStorage Save	Completes the Driver Log module
5	Harvester Management	Core revenue calculation, depends on Driver Logs
6	Machinery Rental	Secondary revenue stream, simpler logic
7	Worker Management	Expense tracking, self-contained
8	Dashboard	Aggregation of all completed modules
9	Reports + PWA Setup	Filtering, grouping, and offline-readiness
10	Export + Final Testing	PDF, WhatsApp share, end-to-end UAT

4.3 Day-wise Execution Plan

Day 1 — Project Setup & Base Structure

Goal: A running React app with the correct folder structure, routing, and reusable base components — nothing more.

8. Run: npm create vite@latest rural-ledger --template react

9. Remove all boilerplate: clear App.jsx content, delete App.css, delete all placeholder content in main.jsx
10. Create the full folder structure exactly as specified in section 3.2
11. Install dependencies: npm install react-router-dom
12. Create common components: Button.jsx, InputField.jsx, TamilLabel.jsx
13. Create placeholder page files: Dashboard.jsx, DriverEntry.jsx, Login.jsx
14. Set up App.jsx with react-router-dom routes
15. Create storage.js with saveData() and getData()
16. Create initialData.js with 3 sample farmers (including F000 Own Farm), 2 drivers
17. Verify: npm run dev loads, routes work, pages render without errors

Day 2 — Driver Entry UI (No Logic)

Goal: The Driver Entry form is pixel-complete on mobile — all fields visible, Tamil labels present, session add/remove works in state only. No calculations, no saving.

18. Machine type dropdown (5 options with Tamil labels)
19. Farmer name dropdown (populated from initialData)
20. Date picker (defaults to today)
21. Sessions section: start time input + end time input per session
22. + Add Session button that appends a new empty session row
23. Remove (X) button per session row
24. Diesel section: radio toggle (Rupees / Litres)
25. Conditional second input for price-per-litre (shown only if Litres selected)
26. Save button (wired to console.log only at this stage)
27. Verify: form renders correctly on a 375px-wide mobile viewport

Day 3 — Driver Logic + Time Calculation

Goal: All calculations work correctly. Sessions are managed in component state. Validation errors are shown.

28. Implement getDuration(start, end) in timeUtils.js with overnight guard
29. Implement getHours(sessions) in calculations.js
30. Wire session state: adding a session appends {start:'', end:'', durationHours:0}
31. On each time input blur, call getDuration and update durationHours for that session
32. Display running total hours below session list
33. Add validation: end time must be after start time (show inline error)
34. Test with: single session, multiple sessions, midnight-crossing session

Day 4 — Diesel Logic + Data Save

Goal: Diesel calculation works in both modes. Completed logs are saved to LocalStorage and persist on reload.

35. Implement getDieselCost(mode, value, pricePerLitre) in calculations.js
36. Wire diesel mode toggle to show/hide price-per-litre input
37. Display computed diesel total below diesel section
38. Implement idGenerator for logs (LOG001, LOG002...)
39. Wire Save button to: validate all fields → compute totals → call addRecord('rl_driver_logs', log)
40. On successful save: clear form, show Tamil success message
41. Add a basic log list below the form showing today's entries
42. Verify: save a log → close tab → reopen → log still visible

Day 5 — Harvester Management Module

Goal: Admin can create a harvester job, link driver logs, and see the bill automatically calculated.

43. Create Harvester.jsx page with job form
44. Farmer selector + season (free-text) + machine type + wet field toggle
45. Rate Per Hour input (admin can override fetched rate)
46. 'Link Driver Logs' section: show all logs for selected farmer + machine type as checkboxes
47. Auto-compute total hours from checked log IDs
48. Display: Gross Amount = hours × rate, Discount input, Final Amount = gross - discount
49. Expense entry section: add diesel, worker, mechanic costs per job
50. Auto-compute net profit = finalAmount - totalExpenses
51. Payment status tracker: paid / partial / pending with amountPaid input
52. Save job to rl_harvester_jobs

Day 6 — Machinery Rental Module

Goal: All three machine rental types work with correct billing units.

53. Create Rental.jsx page
54. Machine selector: Tractor | Plough Tractor | Trailer | Water
55. Auto-set unit type based on selection (tractor/plough→hour, trailer→trip, water→season)
56. Quantity input + Rate Per Unit (fetched from admin pricing config)
57. Auto-display Total Amount = quantity × rate
58. Farmer selector + date + payment status
59. Save to rl_rentals
60. List view of all rental entries with farmer name, machine, amount, status

Day 7 — Worker Management Module

Goal: Worker records and payment entries are saved. Net payable is auto-calculated.

61. Worker list page: show all workers with a '+' button to add entry
62. Work entry form: date, work type dropdown (with 'Other' option), salary inputs
63. Wire calcNetPay(salary, bonus, extra, advance) to display net payable live
64. Show warning if netPayable is negative (over-advance situation)
65. Save work entry to rl_work_entries
66. Worker history view: list all entries for a selected worker with totals

Day 8 — Dashboard Module

Goal: Dashboard shows accurate real-time numbers pulled from all saved data.

67. Read all collections on mount
68. Compute current-month income: sum harvester finalAmounts + rental totalAmounts + own farm harvest
69. Compute current-month business expense: sum expenses where source != 'home_expense' and source != 'lending'
70. Display net profit with colour coding (green positive, red negative)
71. Today's summary: count of driver log entries, total diesel cost
72. Pending lendings count with total outstanding amount
73. Month selector to view any past month's summary

Day 9 — Reports + PWA Setup

Goal: Monthly and machine-wise reports work. App is installable and offline-capable.

74. Monthly report: income/expense/profit breakdown by month, filterable
75. Machine-wise report: group harvester jobs by machine type, show revenue/expense/profit per machine
76. Farmer-wise balance report: outstanding amounts per farmer
77. Install vite-plugin-pwa, configure manifest.json (Tamil app name, icons)
78. Test: install app on Android Chrome using Add to Home Screen
79. Test: turn off WiFi → open app → verify all data is accessible

Day 10 — Export, PDF, WhatsApp & Final Testing

Goal: Data backup and PDF export work. Full end-to-end user flow is tested and verified.

80. JSON Export: 'Backup Data' button serialises all collections to a JSON file download
81. JSON Import: 'Restore Data' file upload input parses and restores all collections
82. PDF Report: Use jsPDF (or similar) to generate a one-page monthly summary PDF
83. WhatsApp Share: Construct a wa.me link with a text summary of the monthly report
84. End-to-end test: full Driver→Harvester→Dashboard flow with real data
85. Usability test with actual driver user; observe and fix confusion points

86. Final polish: Tamil label audit (every field has Tamil), button sizes, loading states

4.4 Coding Standards

Function Naming

Avoid (Over-engineered)	Use (Simple & Clear)
calculateTotalMachineUsageDuration()	getHours()
persistDriverSessionLogEntryToStorage()	saveLog()
retrieveFarmerTransactionHistoryByIdentifier()	getFarmerHistory(id)
computeNetPayableAmountForWorker()	calcNetPay()

Component Rules

- One component per file. Component name matches filename exactly.
- No component file should exceed 150 lines. Break into sub-components if it does.
- Business logic never lives in a component. All calculations go in services/.
- No inline styles. Use a CSS module file alongside each component.
- Comments are brief and describe intent, not the code itself.

PHASE 5 Testing & Quality Assurance Plan

5.1 Testing Objectives

The testing phase ensures that every module functions correctly for its intended real-world use, that data is never lost or corrupted, that all calculations produce exact correct results, and that the interface is usable by non-technical Tamil-speaking users without assistance.

5.2 Unit Test Cases — Calculations

Function	Input	Expected Output	Edge Case?
getDuration()	'09:00', '10:45'	1.75 hours	No
getDuration()	'11:30', '14:00'	2.50 hours	No
getDuration()	'23:00', '01:00'	2.00 hours	YES — overnight
getHours()	Two sessions above	4.25 hours	No
getDieselCost()	rupees, 2000, -	2000	No
getDieselCost()	litres, 10, 102	1020	No
getDieselCost()	litres, 0, 102	0 (show warning)	YES — zero litres
calcRent()	5, 2000, 0	10000	No
calcRent()	5, 2000, 500	9500	No
calcRent()	0, 2000, 0	0	YES — zero hours
calcNetPay()	800, 100, 50, 200	750	No
calcNetPay()	500, 0, 0, 700	-200 (warning shown)	YES — over-advance
calcProfit()	25000, 12000	13000	No

5.3 Integration Test Scenarios

Scenario 1 — Full Harvester Billing Flow

```
1. Driver logs 2 sessions for farmer F001, machineType = harvester_tyre
   Sessions: 09:00-10:45 (1.75h) and 11:30-14:00 (2.5h) → Total: 4.25h

2. Admin opens Harvester Job for F001, season 'Kuruvai 2026'
   Links LOG001 to this job
   Rate: 2300/hr, Wet Field: OFF, Discount: 500
```

```
3. Expected result:
   Gross Amount = 4.25 × 2300 = 9,775
   Final Amount = 9,775 - 500 = 9,275

4. Dashboard for current month should show:
   Income includes ₹9,275
```

Scenario 2 — Worker Over-Advance

```
1. Worker W001 is given advance of ₹700
2. Day's base salary is ₹500, no bonus, no extra
3. calcNetPay(500, 0, 0, 700) = -200

Expected behaviour:
   - Net payable shows: -₹200
   - Warning displayed: 'முன்பணம் அதிகமாக கொடுக்கப்பட்டது'
   - Record saved with netPayable: -200
   - Negative amount treated as ₹200 balance owed TO worker for next entry
```

Scenario 3 — Own Farm Log Flow

```
1. Driver selects 'Own Farm (சொந்த நிலம்)' — farmerId = F000
2. Logs 3 hours on harvester_tyre

3. Admin opens Own Farming expense section
4. Diesel logged separately under Own Farm category

Expected: No harvester billing generated for F000
          Hours appear in rl_driver_logs but no HarvesterJob is auto-created
          Own Farm expenses source-tagged as 'own_farm'
          Dashboard segregates own_farm expenses from business expenses
```

5.4 Offline Testing Protocol

Test	Steps	Expected Result
Cold offline start	Open app, disable WiFi, reload page	App loads from cache; all data visible
Offline data entry	WiFi off → add 3 driver logs → close tab	All 3 logs in LocalStorage
Data persistence	Save data → clear app cache (NOT localStorage) → reopen	Data still present
PWA install	Chrome menu → Add to Home Screen → open from home	Opens as standalone app

	screen	
Backup export	Click 'Export Data' while offline	JSON file downloads successfully
Backup restore	Import previously exported JSON	All collections restored correctly

5.5 Usability Testing — Real User Protocol

Usability testing must be conducted with actual end users before delivery — specifically a real driver and a real farmer from the client's network. The following script is used:

Driver Usability Test

87. Hand the driver the phone with the app open on the Driver Entry screen.
88. Say: 'Please add a machine log for today as you normally would.'
89. Observe silently. Record: hesitations, wrong taps, fields they skip, anything they ask about.
90. Do not help unless they are completely stuck for more than 60 seconds.
91. After completing: ask 'Is anything confusing?' and note response.

Farmer Usability Test

92. Hand the farmer the phone on the Farmer View screen.
93. Say: 'Can you find how much you owe for last month?'
94. Observe and record any confusion with the Tamil labels or navigation.

Any field that causes more than 2 users to hesitate or make an error must be redesigned before delivery — regardless of how the developer believes it should work. Real-world behaviour overrides design assumptions.

5.6 Acceptance Criteria Checklist

#	Criteria	Pass Condition
1	Driver log entry	Completed by driver in < 2 minutes without help
2	Time calculation	All test cases from section 5.2 produce correct output
3	Harvester billing	Rent auto-calculated correctly from linked driver logs
4	Offline persistence	All data survives app close and reopen with WiFi off
5	Dashboard accuracy	Monthly income/expense/profit match sum of individual entries
6	Worker net payable	Formula: (salary + bonus + extra) − advance is correct
7	Farmer view isolation	Farmer A cannot see Farmer B's data

8	Backup / Restore	Export JSON → clear data → import → all data recovered
9	PWA install	App installable and runs from home screen without internet
10	Tamil labels	Every input field has a visible Tamil label

PHASE 6 Deployment & Delivery Plan

6.1 Deployment Strategy

The application is deployed as a static web application hosted on Vercel. Because the system is entirely client-side (no server, no database), the deployment is a simple static file upload. Vercel provides automatic HTTPS, global CDN distribution, and instant deployment from a GitHub repository.

> *Why Vercel (not Play Store): No review process or 48-hour delays. Any update is live in under 60 seconds. Works on any Android browser. HTTPS is automatic. Free tier is sufficient for this application's scale.*

6.2 Build & Deployment Steps

Step 1 — Production Build

```
# From project root
npm run build

# Verify build:
# dist/index.html should exist
# dist/assets/ should contain hashed JS and CSS files
# dist/manifest.json should exist (PWA)

# Local preview (test the built app before deploying):
npm run preview
```

Step 2 — GitHub Repository

```
git init
git add .
git commit -m 'initial: rural ledger mvp'
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/rural-ledger.git
git push -u origin main
```

Step 3 — Vercel Deployment

95. Go to vercel.com → Sign in with GitHub
96. Click 'New Project' → Select your rural-ledger repository

97. Framework Preset: Vite (auto-detected)
98. Build Command: npm run build (auto-detected)
99. Output Directory: dist (auto-detected)
100. Click 'Deploy' — takes approximately 90 seconds
101. Live URL will be: https://rural-ledger.vercel.app (or custom domain)

6.3 Mobile Installation (PWA)

The following steps must be performed by the admin and demonstrated to the client during handover:

102. Open the Vercel URL in Chrome on Android phone
103. Tap the three-dot menu (⋮) in the top-right of Chrome
104. Tap 'Add to Home Screen' (முகப்பு திரையில் சேர்க்கவும்)
105. Name appears as 'கிராம கணக்கு' — tap Add
106. The app icon now appears on the home screen
107. Turn off WiFi. Open the app. Verify it loads and all data is accessible.

After PWA installation, the app behaves like a native mobile application. It opens in fullscreen (no browser address bar), has a Tamil app name, and works completely offline.

6.4 Data Backup & Recovery Strategy

LocalStorage is device-bound and will be lost if the user clears browser data. The following manual backup strategy must be taught to the client during handover.

Backup Type	When	How	Recovery
Weekly JSON Backup	Every Sunday	Tap 'Backup Data' in settings	Upload the JSON file in 'Restore Data'
Pre-Update Backup	Before any app update	Same as above	Same as above
Monthly PDF Archive	First of each month	Generate and WhatsApp the PDF report	For record keeping only

6.5 Client Handover Package

Deliverables Checklist

- Live application URL (Vercel link)
- Tamil user guide document (printed and digital)
- Admin setup guide (how to add farmers, drivers, set prices)

- Backup and restore instructions (with screenshots)
- Developer contact for post-delivery issues (30-day support period)

Tamil User Guide Extract — Driver Entry

```
Driver Entry (ஓட்டுநர் பதிவு) — Step by Step:

1. இயந்திரம் தேர்வு செய்யவும்
   (Select the machine type)

2. விவசாயி பெயர் தேர்வு செய்யவும்
   (Select the farmer's name)

3. தொடக்க நேரம் பதிவு செய்யவும்
   (Enter the start time)

4. முடிவு நேரம் பதிவு செய்யவும்
   (Enter the end time)

5. மேலும் நேரம் சேர்க்க '+' அழுத்தவும்
   (Tap '+' to add another work session)

6. டீசல் தொகை அல்லது லிட்டர் பதிவு செய்யவும்
   (Enter diesel amount in rupees or litres)

7. சேமிக்க 'பதிவு செய்க' அழுத்தவும்
   (Tap 'Save' to save the entry)
```

6.6 Post-Deployment Risks & Mitigations

Risk	Likelihood	Mitigation
Browser storage cleared by user	Medium	Weekly JSON export habit; shown in training
User confusion about Tamil labels	Low	Usability tested pre-delivery; guide provided
Data entry errors by driver	Medium	Validation prevents invalid entries; admin can edit
Device upgrade / data loss	Low	JSON backup portable across devices
App not loading (Vercel outage)	Very Low	PWA cache serves app; Vercel SLA is 99.99%

6.7 Future Phase Roadmap

The following enhancements are explicitly OUT OF SCOPE for this delivery but are documented here for future planning:

Feature	Phase	Complexity	Prerequisite
Cloud sync across devices	Phase 2	High	Backend API + database
Multi-device real-time sync	Phase 2	High	Cloud sync first
OTP-based admin login	Phase 2	Medium	Backend auth service
Farmer WhatsApp notifications	Phase 2	Medium	WhatsApp Business API
Multi-entrepreneur support	Phase 3	High	Full multi-tenant architecture
Advanced analytics / charts	Phase 3	Medium	Data sufficient after 3+ months use
Bank account reconciliation	Phase 3	High	Banking API integration

6.8 Final Project Completion Checklist

The project is considered COMPLETE and ready for delivery ONLY when every item below is verified with a live demonstration.

#	Item	Verified By
1	App is deployed and accessible via Vercel URL	Admin opens URL on phone
2	App is installed as PWA on client's Android phone	Icon visible on home screen
3	App loads and works with WiFi turned off	Live demo without internet
4	Driver can log an entry without assistance	Driver usability test
5	Harvester rent calculates correctly from driver log hours	Test case verified
6	Dashboard shows correct income/expense/profit for current month	Data verified
7	Farmer can view their own records (and only their own)	Live demo with sample farmer
8	JSON backup export and import works	Export → delete data → restore

9	PDF report generates and can be shared via WhatsApp	Live demo
10	All Tamil labels are visible and correctly spelled	Admin review
11	Printed Tamil user guide handed to client	Physical handover
12	Client signs off on delivery	Verbal or written confirmation

---

END OF SDLC DOCUMENTATION

Rural Ledger & Machinery Management System | Version 1.0.0 | April 2026